

Designing a Reliable Local Digital Evidence Architecture

Executive summary

A reliable local digital evidence architecture should be designed first as a **forensic preservation and traceability system**, and only second as a convenience logging or search platform. The architecture's primary goals are to collect evidence in a way that preserves integrity, respects the order of volatility, maintains a strict chain of custody, documents the tools and methods used, and supports later examination and analysis using legally justifiable methods. NIST's forensic guidance and RFC 3227 remain the best high-level anchors: collect from more volatile to less volatile sources, verify exact duplicates cryptographically, and keep documentation sufficient to demonstrate that evidence was not mishandled. ¹

The strongest pattern for such an architecture is a **dual-representation model**: preserve native/raw artefacts exactly as collected, while also generating normalized derivative records for indexing and cross-source analysis. Provenance should be modeled explicitly using the W3C PROV concepts of **Entity, Activity, and Agent**, while CASE/UCO should be used as the investigation-oriented exchange and normalization layer so that results remain traceable to source artefacts, collection actions, tools, operators, and subsequent transformations. Integrity should be layered: content hashing, detached signatures, RFC 3161 trusted timestamps, and an append-only Merkle-based evidence ledger inspired by RFC 9162. Secure transport should use modern TLS with mutual authentication and downgrade-resistant configuration. ²

Because target environments are unspecified, the recommended baseline is **heterogeneous-host capable**: Windows, macOS, Linux, Android, and iOS should each yield native logs, device enumeration records, filesystem and storage artefacts, volatile state when authorized, network captures or socket state, application artefacts, and rich timestamp context. The architecture must also assume that **some local artefacts are mutable, spoofable, incomplete, or policy-altered**. NIST explicitly warns that electronic logs and records can be altered and that organizations must be able to demonstrate integrity. On Windows specifically, driver packages are staged in the Driver Store only after verification and validation, but apparent device metadata can still be affected by INF-based overrides, including monitor EDID overrides stored in the registry. ³

The most important implementation recommendation is therefore this: **treat enrichment as advisory, not evidentiary**. Preserve raw identifiers first, enrich later, and always retain provenance for the enrichment decision. This matters directly for the supplied regression case. The uploaded note describes a DeviceCheck scenario centered on an MSI MAG X870 TOMAHAWK WIFI system and a Realtek USB audio path. `filecite turn0file0` MSI's own specification for that board states it uses a **Realtek ALC4080** codec, while current ALSA UCM configuration maps USB device `0db0:cd0e` to an ALC4080 profile, yet the current `usb.ids` data visible in `hwdata` shows vendor `0db0` but no product `cd0e`. A robust architecture must therefore record the raw VID/PID, preserve device-instance context, and support multi-source enrichment with confidence and provenance instead of collapsing to "unknown device." ⁴

A pragmatic roadmap is: start with a canonical evidence package, strong hashing, provenance, immutable audit trails, and Windows/Linux/macOS host collection; then add memory capture, pcap/pcapng, mobile integrations, and a regression harness modeled after CFTT-style expectations; finally, add higher-assurance tamper evidence, transparency logs, and formal quality metrics tied to incident-response outcomes and lessons learned. NIST's incident lifecycle makes that sequencing sensible: preparation first, then detection/analysis, containment/recovery, and post-incident learning. ⁵

Goals, threat model, and trust assumptions

A reliable local evidence system should optimize for six goals simultaneously: **fidelity, integrity, provenance, reproducibility, operability, and proportionality**. Fidelity means preserving native artefacts in the form in which they were found, including raw images where justified. Integrity means provable non-tampering from acquisition onward. Provenance means being able to answer who collected what, when, where, with which tool, under which policy, and how later derived records were produced. Reproducibility means a second analyst should be able to regenerate the same normalized outputs from the same preserved inputs and tool versions. Operability means the system can actually be used during incident response without excessive fragility. Proportionality means collecting enough for a defensible investigation without unnecessarily over-collecting personal or privileged data. These goals map directly onto NIST's phases of collection, examination, and analysis, and onto CASE's emphasis on traceable investigative actions. ⁶

The threat model should include at least five adversary classes. The first is the **remote attacker** who compromises the host and may alter or delete local artefacts. The second is the **malicious insider or administrator** who has legitimate access but tampers with collection settings, retention, storage, or exports. The third is the **observer or active network attacker** who attempts interception, replay, downgrade, or substitution while evidence is being transported. The fourth is the **tool or pipeline failure mode**: parser bugs, schema drift, silent truncation, bad normalizers, incorrect deduplication, and incomplete enrichment. The fifth is the **legal or governance failure mode**, where evidence becomes unusable because collection was undocumented, retention was wrong, or access controls were insufficiently narrow. These are not hypothetical categories; NIST explicitly notes that logs can be altered, that chain of custody must be clearly defined, and that incident handling must preserve and document evidence through containment and recovery. ⁷

The most consequential trust assumption is that **the endpoint itself is not fully trustworthy once compromise is suspected**. A local collector running with administrator or root privileges can still be lied to by a compromised kernel, hypervisor, or device firmware. That does not make host-side collection useless; it means host-side collection should be treated as one evidence channel among several and should be corroborated with independent sources when possible, such as network captures, remote telemetry, storage images, or externally notarized collection records. RFC 3227's volatility ordering and NIST's treatment of volatile OS data both imply this tradeoff: volatile evidence is valuable, but preserving it safely requires pre-decided criteria and careful handling. ⁸

A second trust assumption is that **timestamps are not self-authenticating**. Host clocks can drift, be misconfigured, be changed by administrators, or be altered by malware. Therefore every collected record should carry at least four temporal fields: the native source timestamp, the collector observation time, the collector clock source and timezone, and any observed skew relative to a trusted reference. Where high

assurance is needed, package manifests and critical transitions should be RFC 3161 time-stamped and later cross-checked against append-only ledger entries. ⁹

A third trust assumption is that **identifier databases are fallible enrichment layers**. Windows device identification strings are meant for comparison and should be treated as opaque strings, not blindly parsed; systemd's USB vendor/model hwdb is imported from `usb.ids`; and public registries such as `usb.ids` or `pci.ids` are useful but not complete. Therefore the architecture should distinguish among: raw identifiers as evidence; vendor or OS metadata as first-party enrichment; public registries as secondary enrichment; and analyst inference as tertiary enrichment. ¹⁰

Local data sources and artefacts

Evidence collection should broadly follow the **order of volatility**: registers and cache first if available, then process/network/kernel state and memory, then temporary filesystems, then disk, then remote logs and topology or configuration context. In practice, most enterprise architectures collect only a subset of this spectrum on every endpoint, but the system should still encode the volatility class of each artefact so analysts understand what was at risk of being lost first. ⁸

The inventory below focuses on **common local artefact families that matter across operating systems**. The central principle is not to force identical collection everywhere, but to preserve a stable set of categories with OS-specific adapters.

Artefact family	Windows	macOS	Linux	Android	iOS	Key normalization fields
Volatile state and memory	Process state, sockets, routing/ ARP, memory when authorized	Process state, sockets, memory only with specialized workflows	Process state, kernel/ device state, memory when authorized	Live shell state via <code>adb</code> , transient logs	Logical acquisition is often more feasible than full physical imaging; access is constrained	<code>volatility_class</code> , <code>capture_method</code> , <code>privilege_level</code> , <code>start_time</code> , <code>end_time</code> , <code>hash</code> , <code>errors</code>
OS and security logs	Event Logging, ETW	Console / unified logging and reports	Audit logs plus kernel-exported state	<code>logcat</code> circular buffers, <code>dumpstate</code> , <code>dumpsys</code>	iOS/mobile workflows are more constrained; use approved device-forensics methods	<code>log_source</code> , <code>channel</code> , <code>record_id</code> , <code>native_timestamp</code> , <code>timezone</code> , <code>boot_id</code> / <code>session_id</code>

Artefact family	Windows	macOS	Linux	Android	iOS	Key normalization fields
Device enumeration and hardware identity	Hardware IDs, compatible IDs, instance IDs, container IDs, USB VID/PID/REV/MI, USBSTOR IDs, Driver Store, monitor EDID and possible INF overrides	Hardware/system reports from System Information; monitor/display metadata	<code>/sys</code> device tree and attributes; udev/hwdb enrichment	USB and device state via <code>adb</code> / bugreport context	Device access is narrower; preserve what authorized tooling can expose	<code>bus</code> , <code>vendor_id</code> , <code>product_id</code> , <code>revision</code> , <code>instance_id</code> , <code>container_id</code> , <code>serial</code> , <code>source_of_name</code> , <code>override_flag</code>
Filesystem metadata and storage images	Bitstream images when needed; preserve filesystem metadata and native artefacts	Preserve native filesystem metadata and reports	Preserve filesystem metadata and block copies where justified	Bugreport may include copied filesystem artefacts under <code>FS/</code>	Logical/backup-style acquisition is often the practical ceiling	<code>image_type</code> , <code>sector_size</code> , <code>filesystem</code> , <code>mount_state</code> , <code>byte_count</code> , <code>hashes</code> , <code>origin_device</code>
Network state and captures	Socket tables, routes, ARP, captures where policy allows	Same categories, with OS-specific tools	Same categories, with OS-specific tools	Bugreport and shell-based diagnostics; captures may require rooted or lab workflows	Usually host-side companion collection rather than device-side pcap	<code>capture_scope</code> , <code>interface</code> , <code>filter</code> , <code>packet_count</code> , <code>clock_basis</code> , <code>loss_stats</code>

Artefact family	Windows	macOS	Linux	Android	iOS	Key normalization fields
Application and browser artefacts	Preserve native application logs, caches, SQLite/JSON/text sources, browser stores	Same pattern; preserve native format first	Same pattern	App logs may also surface through <code>logcat</code> and bugreport	Often constrained to what approved logical acquisition exposes	<code>app_id</code> , <code>artifact_path</code> , <code>native_format</code> , <code>schema_version</code> , <code>acquisition_method</code>

Two Windows-specific points deserve explicit emphasis. First, Microsoft says a hardware ID is a vendor-defined string used to match a device to a driver package, and that Windows also uses compatible IDs, instance IDs, device instance IDs, and container IDs for device management. Second, Microsoft explicitly cautions that device identification strings **should not be parsed** and should be treated as opaque strings for comparison. That implies a good forensic schema must store the raw native string exactly as reported, even if the normalizer also extracts bus-specific fields such as USB VID/PID or PCI VEN/DEV for indexing.

17

Monitor identity is another subtle but important example. Windows allows EDID overrides via INF files, storing override blocks under the device's hardware key in the registry, and those registry values take precedence over EEPROM EDID for the overridden blocks. In evidentiary terms, that means "monitor identity" can have at least two valid layers: raw hardware EDID and OS-effective EDID. A serious architecture should preserve both if available and should flag whether an override was present.

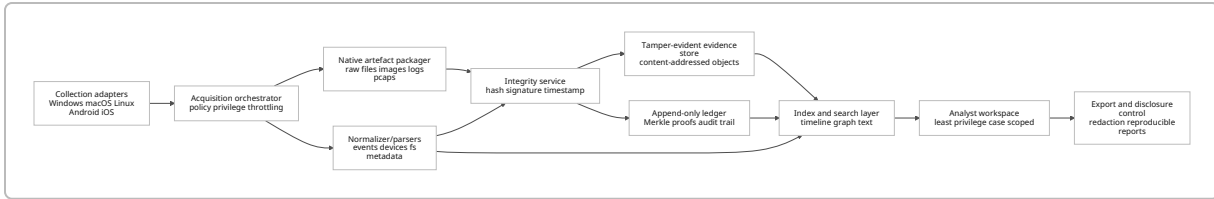
18

For Linux, `/sys` is a first-class evidence source because it exports kernel objects, attributes, and relationships to userspace in a RAM-based filesystem; many device fields are simple ASCII values and can be collected reproducibly. Linux audit data is also important because it provides explicit security/audit records rather than relying solely on general system logs. For macOS, Apple's own documentation confirms that Console collects log messages and compiles reports, while System Information provides structured hardware and model information. For Android, `adb` provides shell access, `logcat` exposes structured circular log buffers, and bugreports can bundle `dumpsys`, `dumpstate`, `logcat`, and copied filesystem content under `FS/`. For iOS and other tightly controlled mobile platforms, NIST's mobile guidance still applies: collect with approved logical or physical methods appropriate to the device and be explicit about what the chosen method cannot reach.

19

Modular architecture

The recommended architecture is modular, evidence-first, and provenance-native. A concise reference design is shown below.



This design aligns with NIST's requirement to preserve integrity and chain of custody, RFC 8446's secure-channel model, RFC 3161's signed timestamping over message imprints, RFC 9162's append-only Merkle log pattern, and CASE/PROV's traceable representation of investigative actions and outputs. ²⁰

Component	Minimum viable design	Higher-assurance design	Why it matters	Sources
Collection adapters	OS-specific scripts or lightweight agents	Signed agents plus boot-media / out-of-band options for high-risk hosts	Host compromise can bias local collection; multiple acquisition modes improve resilience	⁸
Secure transport	TLS 1.3 with server auth	Mutual TLS, certificate pinning, replay protection, short-lived client certs	Evidence transport must resist interception, tampering, and downgrade	²¹
Integrity service	SHA-256/512 hashes and package manifests	Detached signatures plus RFC 3161 timestamps on manifests and exports	Proves exact duplication and package chronology	²²
Tamper-evident storage	Immutable snapshots or append-only object layout	Content-addressed store plus append-only Merkle ledger and external notarization	Prevents silent deletion/rewrite by storage admins or malware	²³
Index and search	Normalized JSON/JSONL with timeline queries	Graph + columnar/event indexes, entity resolution, cross-host correlation	Analysts need search, but native/raw data must remain primary evidence	²⁴
Access control	Role-based access control	Case-scoped ABAC, dual control for exports, privilege separation between collectors and analysts	Limits insider misuse and overexposure of personal data	²⁵
Audit logging	Record package creation, access, export	Signed, append-only audit events linked into evidence ledger	Chain of custody becomes demonstrable, not narrative only	²⁶

Component	Minimum viable design	Higher-assurance design	Why it matters	Sources
Retention and chain of custody	Retention classes and purge jobs	Legal hold, disposition approvals, recorded destruction events, export justifications	Evidence fails in practice when retention and disclosure are informal	27

A key architectural choice is to **separate collection from normalization**. The collector's job is to gather and preserve artefacts with minimal transformation. The normalizer's job is to derive searchable records from those artefacts in a deterministic and repeatable way. This separation matters because Microsoft's device strings may be opaque, EDIDs may be overridden, public ID registries may be incomplete, and log formats may evolve. If you normalize in-place and discard the native source, it becomes difficult to demonstrate what was actually present at collection time. 28

Another important design choice is to model **enrichment as a pipeline with trust tiers**. For hardware identity, for example, the preferred order is: raw bus/device identifiers; first-party OS metadata such as Windows Driver Store contents or monitor hardware key values; curated public registries such as `usb.ids`, `pci.ids`, `pnp.ids`, and systemd hwdb; domain-specific overlays such as ALSA UCM or LVFS firmware metadata; and only then analyst inference. Each enrichment step should emit provenance that records the source, version, and confidence of the mapping. The value of this pattern is demonstrated by the Realtek regression case later in this report. 29

Formats, schemas, provenance, and validation

The architecture should preserve **native formats** wherever possible because native artefacts often carry evidentiary semantics that normalized output cannot fully retain. Recommended preserved-native formats are: raw or forensic container disk images where imaging is justified; native event/log artefacts such as EVTX or exported unified logs; raw memory image formats appropriate to the acquisition tool; `pcapng` for packet capture; SQLite/JSON/text artefacts exactly as found; and raw EDID or firmware metadata blobs. On top of that, produce normalized derivatives in **JSONL or Parquet-like event records** for scalable search, plus **CASE/UCO JSON-LD** for cross-artefact relationships and exchange, and **DFXML-like file metadata records** for filesystem-centric workflows. The standards basis for this recommendation is straightforward: NIST requires documentation sufficient to reproduce and justify collection, PROV defines how entities, activities, and agents relate, and CASE provides an investigation-centric ontology for combination and validation of tool outputs. 30

A practical evidence-package manifest should look conceptually like this:

```
{
  "evidence_package_id": "epkg-2026-06-04-000123",
  "case_id": "CASE-2026-IR-0042",
  "subject": {
    "host_id": "endpoint-7f2a",
    "os_family": "windows",
    "os_version": "11",
```

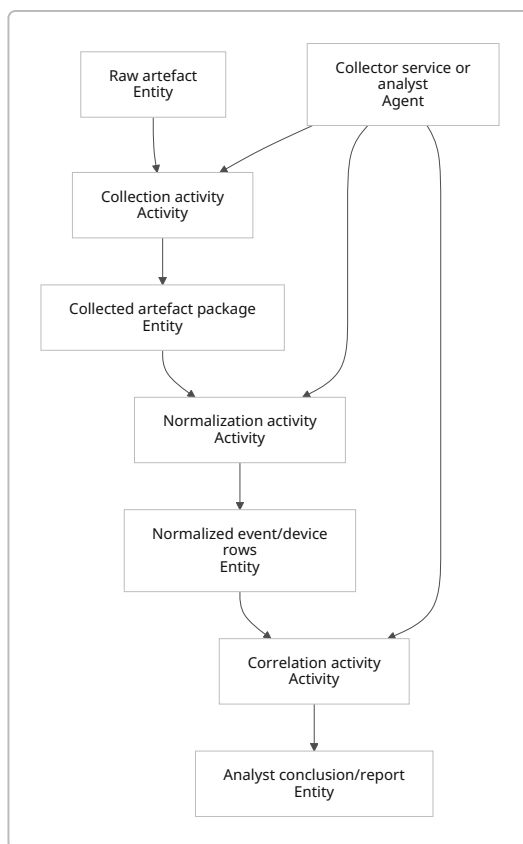
```

    "device_ids": [
      {
        "raw_native": "USB\\VID_0DB0&PID_CD0E&REV_0001",
        "bus": "usb",
        "vendor_id": "0DB0",
        "product_id": "CD0E",
        "instance_id": "...",
        "display_name": "Realtek USB Audio",
        "name_source": "os_driver",
        "enrichment_confidence": 0.66
      }
    ],
    "collector": {
      "tool_name": "collector",
      "tool_version": "1.8.0",
      "build_digest": "sha256:...",
      "operator_id": "svc-evidence",
      "privilege_level": "admin"
    },
    "acquisition": {
      "method": "live_response",
      "started_at": "2026-06-04T08:12:31Z",
      "ended_at": "2026-06-04T08:14:09Z",
      "timezone": "Europe/Athens",
      "clock_source": "system+ntp",
      "observed_clock_skew_ms": 42
    },
    "artifacts": [
      {
        "artifact_id": "art-0001",
        "type": "windows.evtx",
        "native_path": "C:\\Windows\\System32\\winevt\\Logs\\System.evtx",
        "content_hash": "sha256:...",
        "size_bytes": 1234567
      }
    ],
    "provenance": {
      "entity_activity_agent_model": "W3C PROV",
      "exchange_model": "CASE/UCO"
    },
    "integrity": {
      "manifest_hash": "sha256:...",
      "signature": "detached-signature",
      "timestamp_token": "RFC3161"
    }
  }
}

```

The important point is not the exact field names but the **minimum evidentiary content**: raw source identifiers, acquisition context, tool identity, hashes, time information, provenance links, and chain-of-custody transitions. NIST explicitly recommends documenting the hardware and software used during imaging and verifying copied data with message digests; PROV and CASE provide the conceptual model for the rest. ³¹

A useful provenance flow is shown below.



This directly reflects PROV's Entity–Activity–Agent model and is a good fit for CASE serialization when exchanging evidence packages, derived records, and analysis histories. ³²

Validation should be treated as a product surface, not an afterthought. NIST 800-86 explicitly points to the importance of rigorous tool testing and notes the value of automated audit trails and chain-of-custody support; NIST 800-61 in turn makes evidence acquisition, preservation, and post-incident learning part of the response lifecycle. In practical terms, that means every collector, parser, and enrichment source needs **golden corpora, deterministic tests, regression gates, and periodic re-verification** after source or tool updates. ³³

Test class	What it validates	Ground truth / fixture	Pass criteria	Sources
Exact-duplicate acquisition	Disk/log/image copying preserves evidence exactly	Known corpus with reference hashes	Output hashes match; byte count exact; tool/version recorded	34
Order-of-volatility workflow	Volatile artefacts are captured before less-volatile ones where policy requires	Live host simulation with memory + sockets + logs + disk	Collection order conforms to policy; deviations are logged and justified	8
Silent partial-failure detection	Truncated artefacts and incomplete adapters are surfaced	Induced read errors, locked files, interrupted captures	Manifest records partial status; no silent success	25
Timestamp normalization	Clock skew, timezone, boot/session ambiguity are handled	Multi-zone synthetic corpora with skew injection	Native and collection times preserved; skew fields populated	9
Provenance reproducibility	Derived records can be replayed from preserved inputs	Fixed raw artefact set + pinned parser versions	Re-run produces identical normalized output hashes	35
Tamper-evident storage	Rewrite/delete attempts are detectable	Simulated admin tamper, record deletion, backdated inserts	Hash/signature failure or ledger inconsistency is raised	36
Mobile isolation workflow	Collection does not spuriously mutate device state beyond accepted procedures	Controlled Android/iOS fixtures	Isolation method documented; limitations explicit; collected scope matches method	37
Incomplete <code>usb.ids</code> regression	Enrichment does not collapse to false certainty or discard raw IDs when registries are incomplete	Fixture containing vendor <code>0db0</code> , product <code>cd0e</code> , board context, and missing public <code>usb.ids</code> entry	Raw VID/PID preserved; enrichment marked advisory; alternate evidence points to MSI board context and ALC4080 with provenance	38

The Realtek regression case is worth spelling out because it captures the broader design principle. The current `usb.ids` mirror in `hwdata` shows vendor `0db0` as Micro Star International and many product IDs, but no `cd0e`. `systemd`'s USB `hwdb` is imported from `usb.ids`, so Linux enrichment will inherit the same omission unless supplemented. At the same time, ALSA's `alsa-ucm-conf` explicitly maps

`0db0:cd0e` to the **Realtek/ALC4080** profile, and MSI's board specification for the MAG X870 TOMAHAWK WIFI lists a **Realtek ALC4080 Codec** with USB high-performance audio. The correct regression behavior is therefore: preserve the raw identifier; mark the direct `usb.ids` lookup as incomplete; admit stronger contextual evidence from ALSA and board specification; record the provenance and confidence of the inferred mapping; and never overwrite the raw identifier with a single "pretty name" as if it were primary evidence. ³⁹

For Windows patch and firmware context, two enrichment sources are particularly useful but should still be treated as secondary metadata. First, Windows Update Agent supports offline scanning using the Microsoft-signed `wsusscn2.cab`, which is safer and more defensible than brittle web scraping of catalog pages. Second, LVFS metadata can provide firmware GUIDs, release metadata, and integrity annotations useful for device/firmware correlation on supported systems. Both are valuable for reproducible evidence context, but neither should replace the underlying raw artefacts from the endpoint. ⁴⁰

Deployment, operations, legal, and resilience

Deployment should assume **mixed trust zones and mixed connectivity**. Some endpoints will support resident agents; some should be handled by just-in-time live response over secure channels; some high-risk systems should be collected using bootable external media or other out-of-band workflows; and mobile devices frequently require specialist workstations and lab procedures rather than broad autonomous collection. As with NIST's incident lifecycle, preparation is the determinative phase: privilege models, collection playbooks, volatile-data decision criteria, and evidence-handling rules must be established **before** the incident, not improvised during it. ⁴¹

For scale, use a **tiered storage model**. Keep recent cases and active investigations in a hot tier with low-latency search. Move verified native artefacts to a warm immutable store. Move long-retention packages to a cold tier with the ledger, hashes, signatures, and timestamps still online. Search indexes should be rebuildable from preserved packages; they are analytical conveniences, not evidence of record. That design sharply reduces the risk that an index migration, parser bug, or retention misconfiguration silently destroys the authoritative evidence set. The same principle is implied by NIST's recommendation to avoid doing forensics on the evidence copy itself and instead work from a reproducible duplicate. ⁴²

Operational maintenance should include **schema versioning, parser pinning, source-manifest versioning, and signer rotation**. Public enrichment sources change over time: `hwdata` has frequent releases; `usb.ids` changes; ALSA UCM evolves; firmware metadata changes; and Windows documentation and metadata sources evolve as well. Therefore each package or normalization run should record not just "what source was used," but its exact release, commit, version date, or digest. `hwdata`, for example, publishes frequent releases and contains `pci.ids`, `usb.ids`, and `pnpi.ids`; `alsa-ucm-conf` is separately versioned and validated; and `fwupd` /LVFS metadata has its own schema and lifecycle. ⁴³

Incident-response integration should be explicit. Every evidence package should carry a case or incident identifier, severity, triggering alert or task reference, and handling phase. During containment, eradication, and recovery, evidence acquisition and preservation should be first-class actions, not side notes, and post-incident lessons learned should feed directly into updated playbooks, new regression tests, and revised

retention or disclosure policies. NIST states plainly that lessons learned are often omitted even though they are one of the most important parts of incident response. ⁴⁴

Legally and regulatorily, the architecture should be opinionated about process but humble about jurisdiction. NIST 800-86 explicitly says organizations are subject to different laws and regulations and that the guide should not be treated as legal advice. The practical implication is that the platform must support **policy-mappable controls**: data minimization profiles, privileged-material marking, works-council or employee-notice variants, legal hold, export review, redaction logs, and disposition documentation. The safest default is to minimize routine collection to high-value categories, then elevate to deeper acquisition only under a documented case rationale. ⁴⁵

The principal failure modes and corresponding recovery strategies are predictable. **Silent partial collection** should be countered by explicit completeness metadata and fixture-based negative testing. **Transport replay or substitution** should be mitigated with mutual TLS, nonce-bound uploads, and signed manifests. **Store-side tampering or deletion** should be countered with immutable storage, append-only ledgers, and off-system notarization. **Clock drift or manipulation** should be mitigated by storing multiple time references and using trusted timestamps on package manifests. **Parser or schema drift** should be mitigated by replayable normalization and corpus regression gates. **Source-db poisoning or incompleteness** should be mitigated by trust-tiered enrichment and provenance on every mapping. **Collector compromise** should be mitigated by corroboration, higher-assurance acquisition modes, and clear analyst confidence labels. ⁴⁶

Prioritized roadmap and success metrics

The implementation roadmap should be driven by evidentiary value, not feature count.

First priority is to establish the **canonical evidence package**: native artefact preservation, manifest schema, strong hashes, tool/version capture, basic chain-of-custody records, and a storage layout that can enforce immutability. Without that layer, later analytics only create faster uncertainty. This phase should also deliver Windows/Linux/macOS host adapters for core artefacts: system/security logs, device enumeration, filesystem metadata summaries, network state, and selected application logs. ⁴⁷

Second priority is reliable **normalization and indexing**. Adopt a stable event/device schema, PROV/CASE-based provenance, deterministic parsers, and a search layer that can be rebuilt from preserved packages. This is where cross-source timelines and entity resolution start to become genuinely useful, especially for device attribution and host activity reconstruction. ⁴⁸

Third priority is **high-value but failure-prone acquisition**: memory capture, packet capture, mobile workflows, and hardware/firmware enrichment. This phase should not begin until the regression harness exists, because these capabilities are where silent corruption, incompatibility, and analyst overconfidence tend to spike. The Realtek `usb.ids` regression case belongs here as a permanent test, not as a one-time workaround. ⁴⁹

Fourth priority is **tamper-evident assurance and governance maturity**: RFC 3161 timestamping, append-only evidence ledgering, off-system notarization, legal hold, export review, disclosure/redaction

workflows, and destructive-disposal recording. This is the phase that turns a technically competent repository into a defensible evidentiary system. ⁵⁰

Success should be measured with a small set of hard metrics rather than a broad dashboard. Recommended metrics are:

- **Collection success rate:** percentage of scheduled or case-triggered collections that complete with no unacknowledged partial failures.
- **Integrity verification rate:** percentage of artefacts and packages whose hashes, signatures, and timestamps validate on ingest and during periodic re-verification.
- **Provenance completeness:** percentage of normalized records that can be traced back to a preserved native artefact, collection activity, tool version, and operator/service identity.
- **Reproducibility rate:** percentage of normalization jobs that reproduce byte-identical derived outputs when rerun on preserved inputs with pinned tool versions.
- **Time-to-evidence:** median time from incident declaration to first searchable, integrity-verified package available to analysts.
- **False-unknown device rate:** percentage of device records with preserved raw IDs but unresolved names after multi-source enrichment; this should drop over time without sacrificing provenance or confidence labeling.
- **Export defensibility:** percentage of case exports that include complete chain-of-custody history, integrity materials, and scope justification.
- **Recovery drill success:** percentage of quarterly drills in which a case can be reconstructed from preserved packages and ledger/audit records alone.
- **Lessons-learned closure:** percentage of post-incident action items that produce a control, playbook, or regression-test change within the target window. ⁵¹

Open questions and limitations

Some areas remain environment-dependent or require organization-specific decisions. The exact **retention schedule**, legal-hold triggers, and privacy boundaries cannot be prescribed from standards alone because NIST explicitly notes that laws and regulations vary by organization and jurisdiction. ⁵²

macOS and especially iOS artefact coverage is also more variable than Windows or Linux because platform protections and collection permissions differ sharply by device state, tooling, and authorization model. NIST's mobile guidance supports the general acquisition strategy used here, but precise artefact reach should be validated in your own lab before any claims are made about completeness. ⁵³

Finally, public third-party enrichment sources should be reviewed under your own legal and supply-chain process before bundling or redistributing them. In this report, `hwdata`, `usb.ids`, `pci.ids`, ALSA UCM, fwupd, LVFS, and Microsoft-signed WUA metadata were identifiable and useful from accessible primary sources, but some other third-party ecosystems were not sufficiently verifiable from authoritative documentation during this review to recommend for automatic bundling without further counsel and source review. ⁵⁴

2 32 35 48 <https://www.w3.org/TR/prov-overview/>
<https://www.w3.org/TR/prov-overview/>

3 5 6 7 14 16 20 22 25 26 27 30 31 33 34 45 47 52 <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf>
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-86.pdf>

4 <https://www.msi.com/Motherboard/MAG-X870-TOMAHAWK-WIFI/Specification>
<https://www.msi.com/Motherboard/MAG-X870-TOMAHAWK-WIFI/Specification>

9 50 <https://www.rfc-editor.org/rfc/rfc3161>
<https://www.rfc-editor.org/rfc/rfc3161>

10 28 <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/device-identification-strings>
<https://learn.microsoft.com/en-us/windows-hardware/drivers/install/device-identification-strings>

12 <https://learn.microsoft.com/en-us/windows/win32/eventlog/event-logging>
<https://learn.microsoft.com/en-us/windows/win32/eventlog/event-logging>

13 17 <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/hardware-ids>
<https://learn.microsoft.com/en-us/windows-hardware/drivers/install/hardware-ids>

18 <https://learn.microsoft.com/en-us/windows-hardware/drivers/display/overriding-monitor-edids>
<https://learn.microsoft.com/en-us/windows-hardware/drivers/display/overriding-monitor-edids>

19 <https://docs.kernel.org/filesystems/sysfs.html>
<https://docs.kernel.org/filesystems/sysfs.html>

21 46 <https://www.rfc-editor.org/rfc/rfc8446>
<https://www.rfc-editor.org/rfc/rfc8446>

23 36 <https://www.rfc-editor.org/rfc/rfc9162>
<https://www.rfc-editor.org/rfc/rfc9162>

24 <https://caseontology.org/>
<https://caseontology.org/>

29 <https://learn.microsoft.com/en-us/windows-hardware/drivers/install/driver-store>
<https://learn.microsoft.com/en-us/windows-hardware/drivers/install/driver-store>

37 53 <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-101r1.pdf>
<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-101r1.pdf>

38 <https://raw.githubusercontent.com/vcrhonek/hwdata/master/usb.ids>
<https://raw.githubusercontent.com/vcrhonek/hwdata/master/usb.ids>

39 <https://usb-ids.gowdy.us/>
<https://usb-ids.gowdy.us/>

40 https://learn.microsoft.com/en-us/windows/win32/wua_sdk/using-wua-to-scan-for-updates-offline
https://learn.microsoft.com/en-us/windows/win32/wua_sdk/using-wua-to-scan-for-updates-offline

41 44 51 <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>
<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-61r2.pdf>

43 54 <https://github.com/vcrhonek/hwdata>
<https://github.com/vcrhonek/hwdata>